

캡스톤 디자인 프로젝트

최종 발표

MIML(MusicIsMyLifeMIML)

단국대학교 소프트웨어학과
32232779 유소현
32234852 한주연

목차

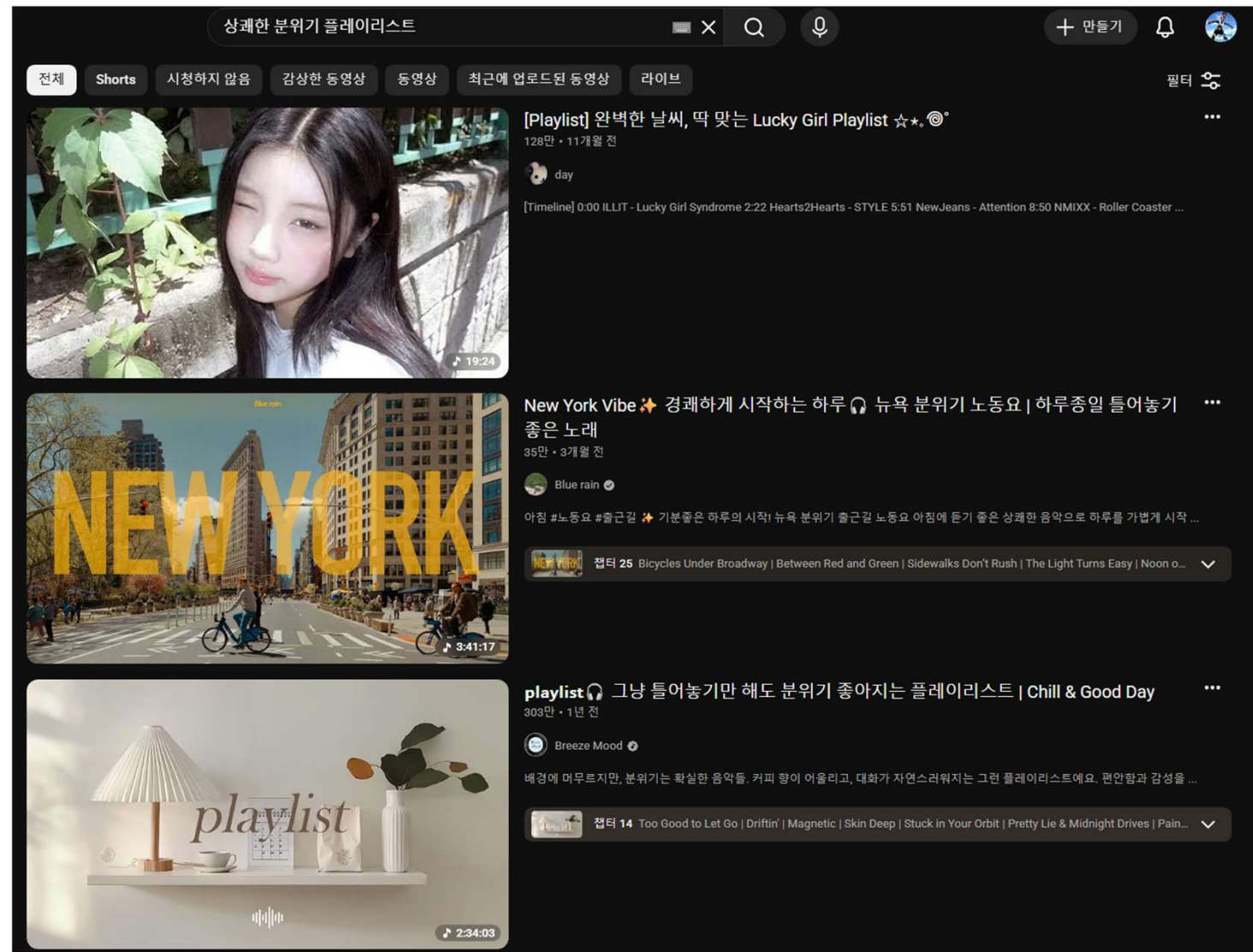
1. 개발 배경 및 동기
2. 시스템 아키텍처
3. 핵심 기능 구현
4. 구현 결과
5. 결론
6. 시연 영상

1. 개발 배경 및 동기

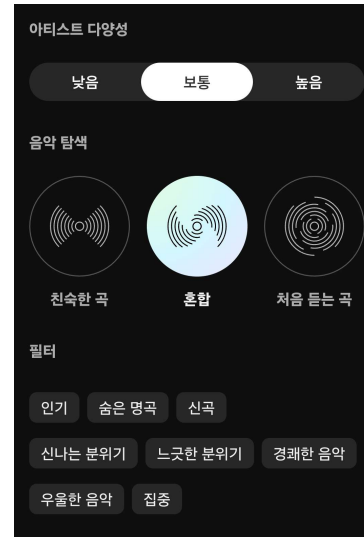
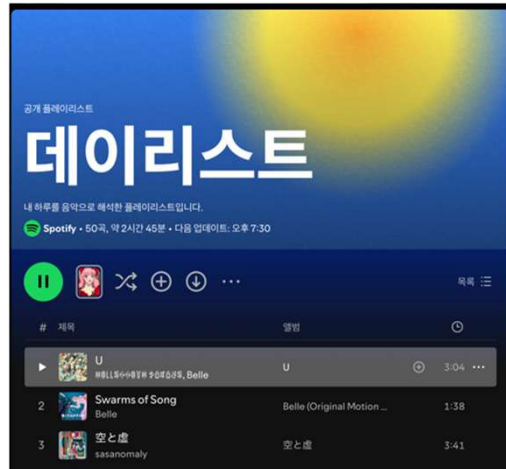
유튜브 감성 플레이리스트 채널
누적 조회수 수억 건 → 상황·분
위기 기반 음악 수요 확인

기존 스트리밍 서비스:
장르·아티스트 추천 위주 →
"지금 내 감정에 맞는 곡" 찾기
어려움

자연어 입력 → AI 오디오 분석
으로 맥락 기반 음악 자동 추천
서비스 개발

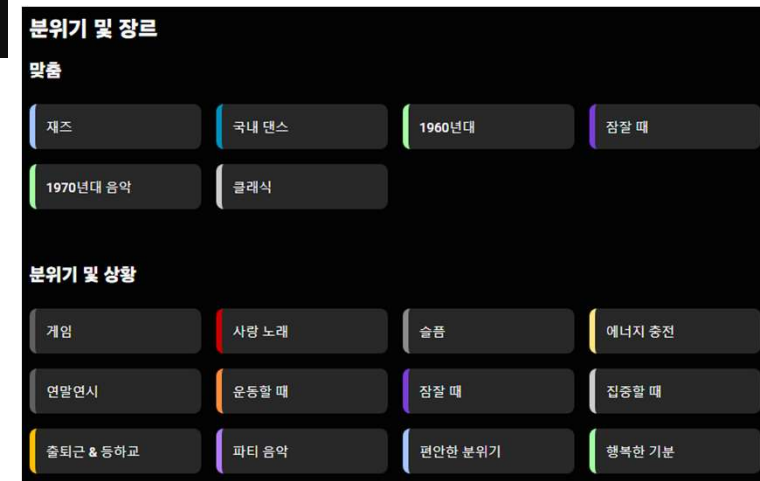


1. 개발 배경 및 동기-기존 음악어플의 한계



- 하루 단위로 생성되므로 사용자가 원하는 시점 맞춤 추천 제공 불가
- 사용자 데이터가 충분히 축적되기 전 까지 개인화 수준이 제한적

- 분위기를 판단하기에 부족한 필터 개수
- 개인 취향이 반영되지 않은 정형화된 분위기 및 장르 기반 추천



1. 개발배경 및 동기-개발 목표

1. 자연어 기반 감성 인식 및 하이브리드 추천

- GPT로 텍스트에서 오디오 피처·감성 태그를 추출하고, SoundNet 오디오 분석 + 유클리드 거리 기반 오디오 매칭으로 분위기에 맞는 음악 추천

2. 개인 맞춤형 추천 및 피드백 반영

- 사용자 청취 이력으로 취향 프로필을 구축하고, 좋아요/싫어요 피드백으로 추천 개인화

3. 커뮤니티 기반 태그 생태계

- 사용자가 직접 감성 태그를 등록하여 추천 품질을 집단지성으로 개선하고 음악 경험을 공유

2. 시스템 아키텍처

기술 스택

Backend

Kotlin + Spring Boot 3

Spring Security + Firebase Auth

JPA / Hibernate + MySQL (AWS RDS)

음원 분석

SoundNet API — energy, happiness, danceability 등 14개 피처

External APIs

Spotify API — 곡 메타데이터, 아티스트 검색

OpenAI GPT-4o-mini — 감성 태그 생성 및 자연어 처리

Last.fm API — 아티스트 장르 조회

Infra

AWS EC2 (Amazon Linux 2023, Java 17)

AWS RDS — MySQL 8

Firebase — 인증 + Firestore

2. 시스템 아키텍처

AWS 인프라 구성

EC2 서버

Amazon Linux 2023, Java 17 (Corretto)

Spring Boot JAR

탄력적 IP 고정 할당

RDS 데이터베이스

MySQL 8, ap-northeast-2

EC2 보안 그룹 기반 접근 제어

Firebase

Firebase Auth — 모든 API 엔드포인트 보호

Firestore — 플레이리스트, 커뮤니티 글 저장

MIML 시스템 아키텍처

Music Is My Life · 기술 스택 & 컴포넌트 상호작용

클라이언트

백엔드 (AWS EC2)

데이터 저장소

Spotify API

음악 메타데이터 · 검색

Last.fm API

아티스트 장르 조회

SoundNet API

오디오 피쳐 14종 분석

OpenAI GPT-4o-mini

태그 생성 · 추천 멘트

안드로이드 앱

React

- Firebase SDK 직접 연동
- REST API 호출 · 인증

REST API

EC2 Spring Boot 서버

Kotlin · Spring Boot 3 · Spring Security · JPA / Hibernate

- TagGenerationService
GPT 감성 태그 생성
- SmartRecommendationService
개인화 음악 추천
- MusicAnalysisService
SoundNet 피쳐 분석
- SpotifyClient
음악 메타데이터 수집
- LastFmClient
아티스트 장르 조회
- Firebase Auth 검증
JWT 토큰 파싱

REST API · Spring Security Filter · Firebase Auth Token 검증

/api/recommend/smart /api/tags/generate /api/feedback/* /api/playlists

인증

DB 조회 / 저장

AWS RDS MySQL

MySQL 8 · JPA/Hibernate

- music · audio_features
- music_tags · artist_genres
- users

Firebase

Auth + Firestore

- 사용자 로그인 · 인증
- JWT 토큰 발급
- 플레이리스트 저장

Firebase

Auth + Firestore

- 인증 토큰 관리
- 플레이리스트 저장
- 커뮤니티 글 저장

토큰 검증 · 저장

안드로이드

Spring Boot

MySQL RDS

Firebase

OpenAI

Spotify

SoundNet

Last.fm

2. 시스템 아키텍처

데이터베이스 설계

users

Firebase UID 기반 유저 정보 저장. 앱 로그인 시 자동 생성되며 추천 프로필의 주체가 됨.

Music

Spotify에서 가져온 곡 메타데이터 (제목, 아티스트, 앨범, 앨범아트 등) 저장. 추천의 기본 단위.

audio_features

각 곡의 템포·에너지·댄서빌리티·발랄함 등 Soundnet 오디오 수치 저장. 사용자 감성 입력과의 매칭에 사용됨.

music_tags

GPT가 생성한 감성 태그와 커뮤니티 유저들이 작성한 리뷰에서 추출한 태그를 함께 저장. 추천 시 텍스트 기반 태그 점수 산출에 활용됨.

artist_genres

Last.fm에서 가져온 아티스트별 장르 정보 저장. "케이팝 추천해줘" 같은 장르 필터링의 핵심 데이터.

satisfaction_feedback_log

곡별 좋아요/싫어요 피드백 저장. 유저 취향 프로필을 점진적으로 학습하는 데 사용됨.

2. 시스템 아키텍처

음악 데이터 수집 파이프라인

1. Spotify API — 아티스트별 인기곡 검색 (artist: 검색으로 정확도 확보)
2. SoundNet API — Spotify ID 기반 오디오 피쳐 분석 (14개 수치)
3. music + audio_features 테이블 동시 저장
4. GPT-4o-mini — 오디오 피쳐 기반 감성 태그 자동 생성 → music_tags 저장 (source=GPT)
5. Last.fm API — 아티스트 장르 조회 → artist_genres 저장 (중복 아티스트 스킵)

3.핵심기능구현

자연어 기반 추천 알고리즘

1. 사용자 자연어 입력 → GPT-4o-mini 분석
오디오 피쳐 수치 (energy / happiness / danceability / acousticness / tempo) + 감성 태그 2~4개 + 장르 필터 + 추천 멘트 동시 생성
2. 유저 취향 프로필과 블렌딩 (profileRatio 슬라이더 0.0~1.0)
3. DB 오디오 범위 필터 → 후보 풀 추출 (± 35 범위, 상위 limit $\times$ 6곡)
4. 태그 코사인 유사도 커버리지 + 인기도 보정으로 최종 순위 결정
최종 정렬 전 ± 0.08 랜덤 노이즈 주입 → 동점권 곡 순서 매 요청마다 변화
5. 아티스트 중복 방지 (가수당 최대 1곡) + $\text{score}^{0.5}$ temperature 적용 가중치 확률 샘플링

3.핵심기능구현

추천 알고리즘 변화

1. 코사인 유사도 방식의 한계

설계서대로 오디오 피처에 코사인 유사도 적용 시, '방향이 비슷한 곡'만 추천되는 문제 발생. 피처 수치 간 절대 차이가 중요함을 확인 후 유클리드 거리 기반으로 교체

2. 특정 아티스트 추천 독점 현상

실제 테스트 시 점수 0.93~0.96에 동일 아티스트 곡들이 상위권을 독식 (Radiohead, IU 등 반복 추천).
가수당 최대 1곡 제한 + $\text{score}^{0.5} \text{ temperature} + \pm 0.08 \text{ jitter}$ 도입으로 해결

3. 태그 점수 산식 재설계

설계서의 `relevance_score` 단순 합산은 태그 수가 많은 곡이 무조건 유리한 편향 발생. 요청 태그 ↔ DB 태그 코사인 유사도 커버리지로 교체해 매칭 정확도 개선

3. 핵심 기능 구현

추천 점수 계산

오디오 점수

5개 피쳐 축 (energy, happiness, danceability, acousticness, tempo)
유클리드 거리 → 0~1 정규화
최소 임계값 0.45 미달 시 후보 제외

장르 필터

genreInclude 명시 시 해당 장르 아티스트만 통과
artist_genres 테이블 기반 — 데이터 없는 아티스트도 제외

태그 유사도 점수

요청 태그 ↔ DB 태그 코사인 유사도 커버리지
voteCount 보정 최대 +20%

최종 점수 산식

태그 있을 때: 태그점수 × 0.85 + 인기도 × 0.15
태그 없을 때: 오디오점수 × 0.85 + 인기도 × 0.15

아티스트 다양성

동일 가수 최대 1곡 제한
가중치 확률 샘플링 시 $\text{score}^{\wedge 0.5}$ temperature 적용
(점수 차이 압축 → 하위권 곡도 선택 기회 확보)
최종 정렬 전 ± 0.08 랜덤 노이즈 주입
(동점권 순서 매 요청마다 변화)
후보 풀 확대: 상위 limit × 6곡 으로 다양한 아티스트 포함

3.핵심기능구현

추천 API 응답 구조 POST /api/recommend/smart

Request Body

description 자연어 상황 설명 (string)

profileRatio 0.0=현재 분위기 / 1.0=취향 (double)

limit 추천 곡 수 (default 10)

Response

message GPT 생성 자연어 추천 멘트

moodFeatures 분석된 오디오 피쳐 수치

recommendations 추천 곡 목록

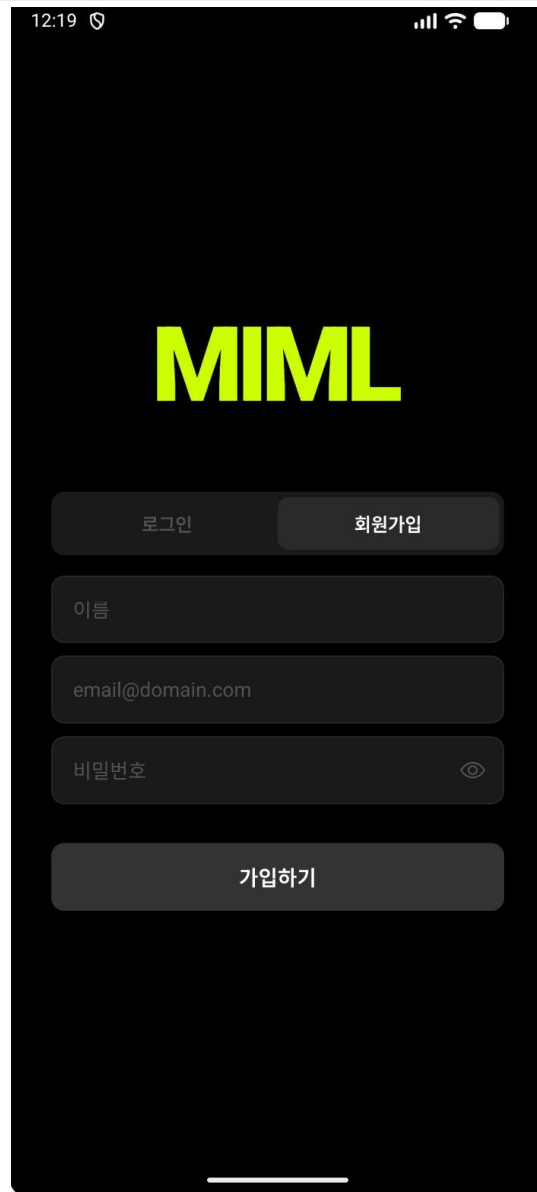
musicId, title, artist

albumImageUrl, spotifyId

score (0.0 ~ 1.0)

totalCandidates / returnedCount

4. 구현 결과 -로그인화면



12:19

MIML

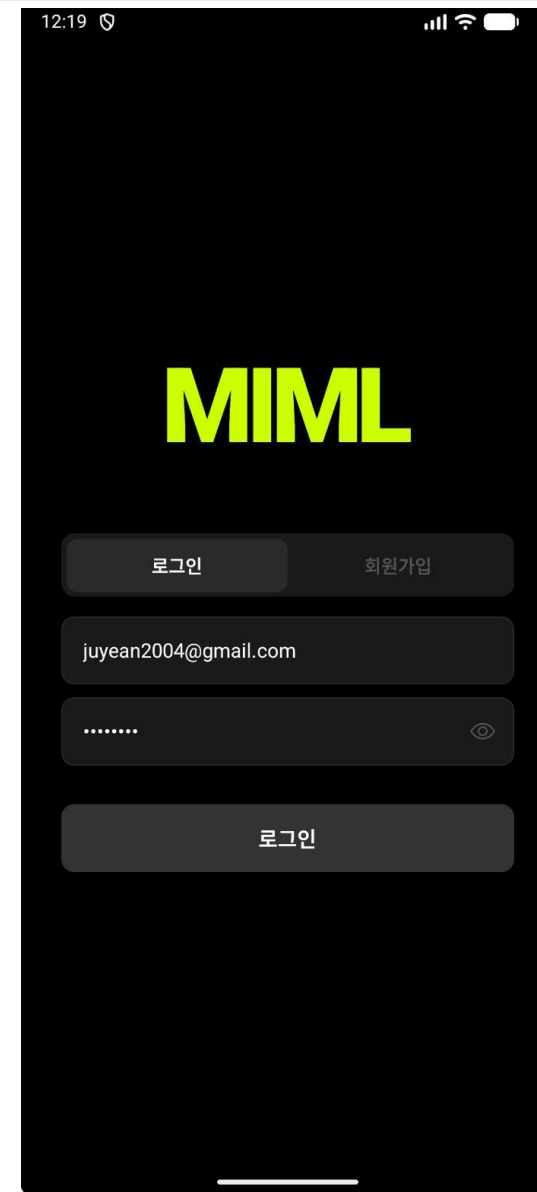
로그인 회원가입

이름

email@domain.com

비밀번호

가입하기



12:19

MIML

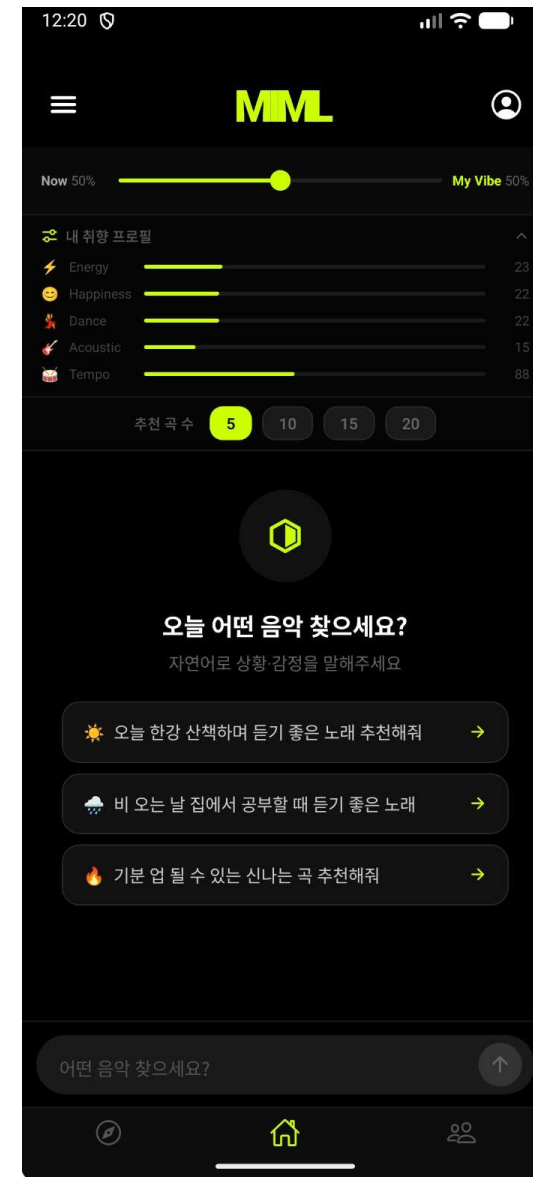
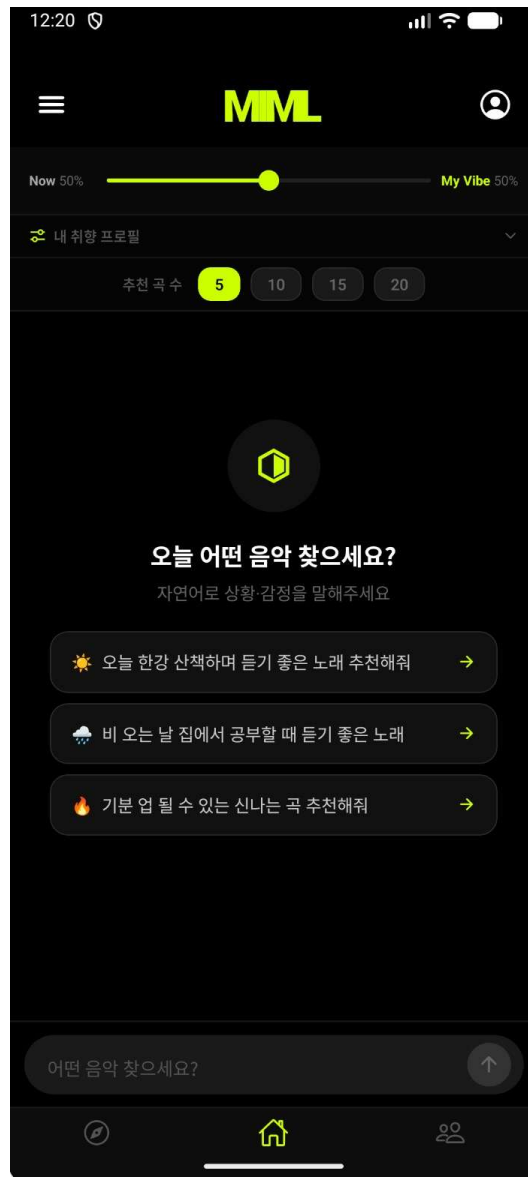
로그인 회원가입

juyean2004@gmail.com

.....

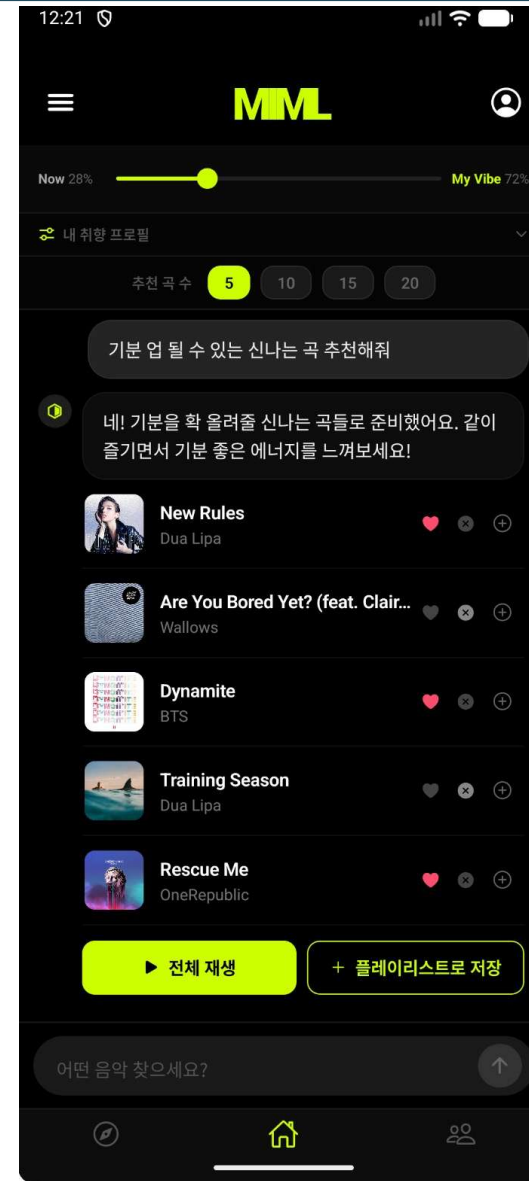
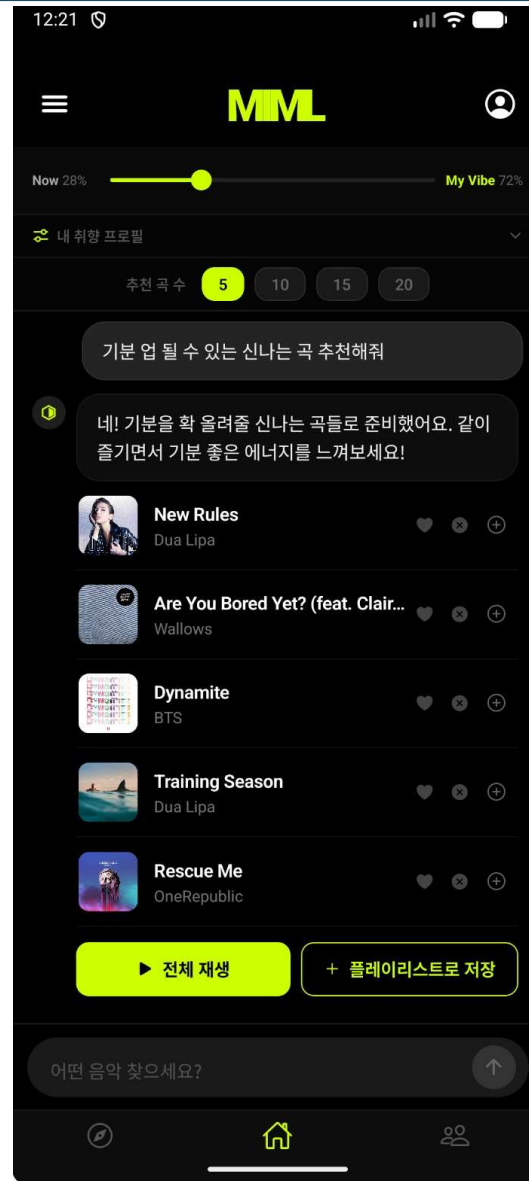
로그인

4. 구현 결과 -홈 화면

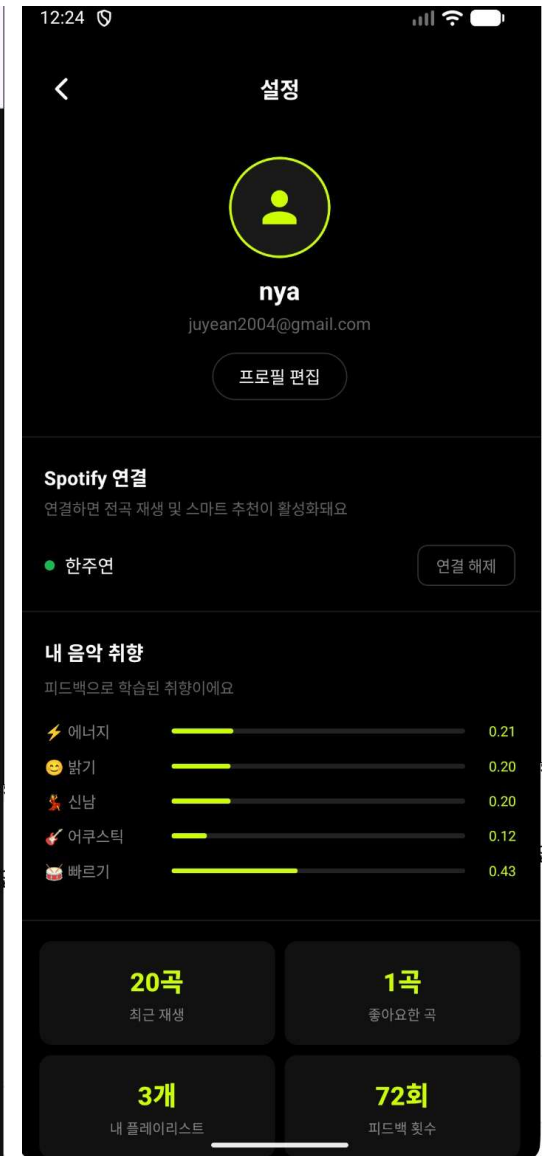
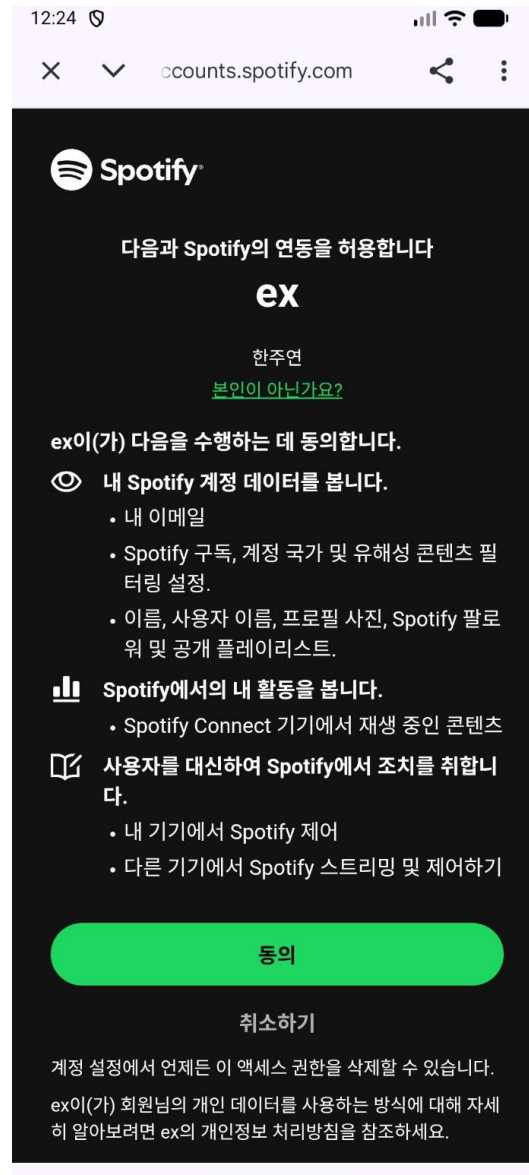
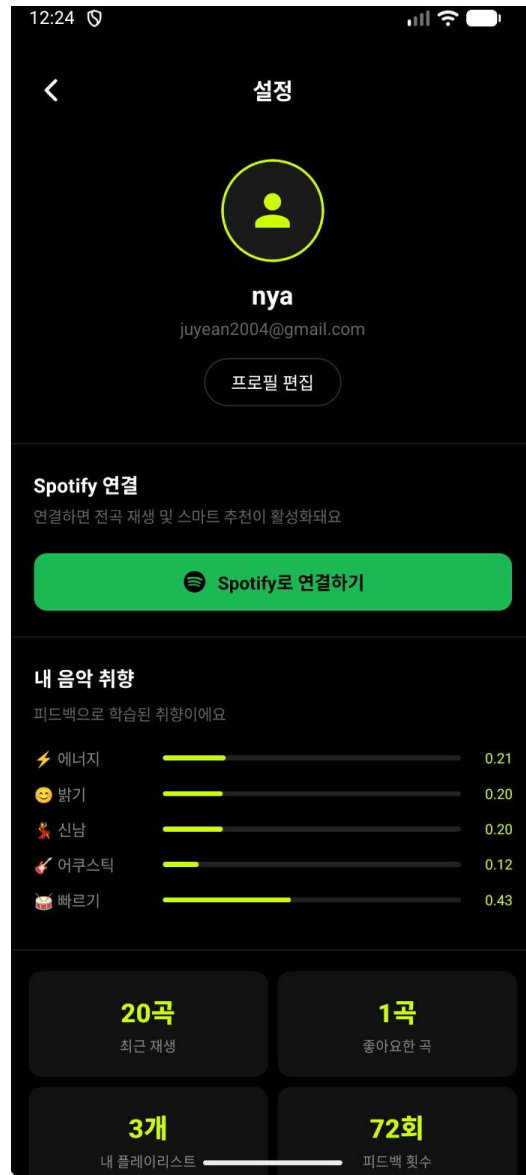


4. 구현 결과

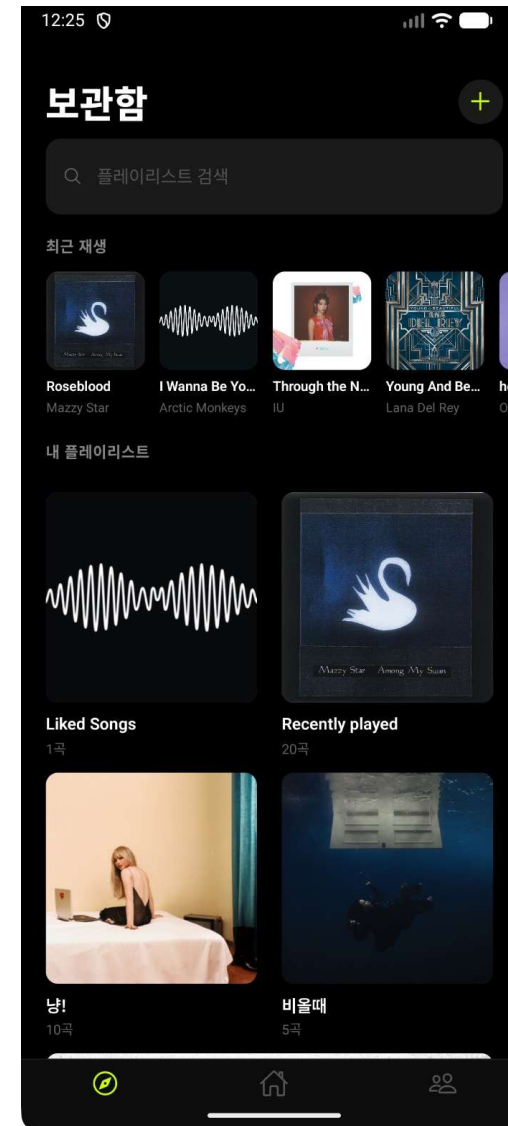
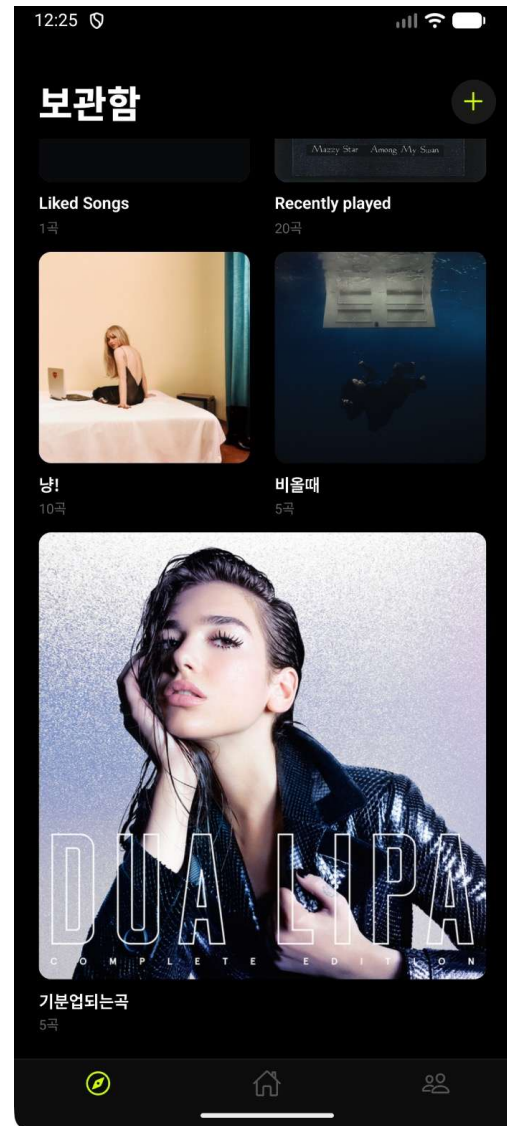
- 추천 결과 화면



4. 구현 결과 -사용자프로필



4. 구현 결과 -플레이리스트 보관함



5. 결론

구현 목표 달성 – 핵심 성과

1. 자연어 → 오디오 피쳐 + 감성 태그 동시 추출

자연어 입력 한 문장으로 오디오 수치 5개 + 감성 태그 + 장르 필터 + 추천 멘트를 동시 생성, 맥락 기반 추천 실현

2. 오디오 수치 + 커뮤니티 태그 하이브리드 추천

SoundNet 오디오 피쳐(14개 수치)와 GPT·유저 리뷰 태그를 함께 활용 — 기존 서비스가 하지 못한 이중 레이어 추천 구조 구현

3. 현재 분위기 ↔ 평소 취향 실시간 블렌딩

profileRatio 슬라이더로 사용자가 직접 추천 비율 조절 가능 — 같은 입력도 상황에 따라 다른 결과 제공

4. 아티스트 다양성 메커니즘

단순 점수순 정렬의 한계(특정 아티스트 독점)를 score temperature + jitter + 가수당 1곡 제한으로 극복

5. 피드백 기반 적응형 프로필 학습

좋아요/싫어요 피드백 시 $\alpha = 1/(n+10)$ 감쇠 학습률로 profile_vector 즉시 업데이트 — 누적될수록 안정적으로 수렴하는 개인화 설계

5. 결론

한계 및 향후 개선 방향

1. GenreAliases · TagVocabulary 하드코딩

17개 장르 / 92개 감성 태그가 코드에 고정 — 신규 장르·태그 추가 시 매번 재배포 필요. 향후 DB 테이블화 + Admin API로 전환해 배포 없이 동적 관리 가능한 구조로 개선 필요

2. 장르 필터 시 추천 후보 희소성

특정 장르 지정 시 DB 내 해당 장르 곡이 부족해 다양성 저하.

artist_genres 커버리지 확대 및 지속적인 데이터 확충이 서비스 품질의 전제 조건

3. 재추천 버튼 구현

추천 결과에 대한 사용자 피드백을 즉각 수용하여 audio 특성을 수정한 다음 재추천 수행

감사합니다